

Portfolio Building

→ Team Name : APEX GIP

→ Team Members:

(i) Lead Developer - M. Sampriyan
(R25EJ060)

(ii) Lase Engineer - Dilip Naidhruwa
(R25EJ029)

(iii) Documentation - Shrirama Venkat
- araman

→ Problem Statement/ Understanding :

* Designing a line editor that should operate using terminal commands.

* It should completely work on a terminal using predefined instructions to perform the corresponding

* Designing a data structure that holds the document line in memory using dynamic memory allocation

→ Problem Analysis & Code design :

(i) Data Structure - Array of strings

(ii) Commands Used - INS, DEL, EDIT, SHOW, SAVE, LOAD, FIND, REPLACE, STATS, UNDO, CLEAR, EXIT.

→ Execution :

Algorithm of Program flow :

- Step 1 - Create & Initialize Memory
- Step 2 - Print & display Instructions
- Step 3 - Read input string
- Step 4 - Check for Instruction Keyword
- Step 5 - Evaluate & Execute code based on input & instruction using 'Else-if' loop

HELP.md X

HELP.md > abc # Simple Line Editor (Minimal Edition) — Help

You, 2 weeks ago | author (You)

1 # Simple Line Editor (Minimal Edition) – Help

You, 2 weeks ago • Add

2

3 A command-line line editor. Type a command name (case-insensitive) at
4 the `editor>` prompt, followed by any arguments. Line numbers are
5 ****1-indexed**** (the first line is line 1). Text matching inside the
6 document (FIND, REPLACE) is ****case-sensitive****, even though the command
7 names themselves are not.

8

9 Type `HELP` at any time inside the editor to see the command list again.

10

11 ****Limits:**** up to 500 lines, up to 255 characters per line.

12

13 ---

14

15 ## Commands

16

17 ### `SHOW` – Display the document

18 Prints every line with its line number.

19

20 ```

21 editor> SHOW

22 --- DOCUMENT START ---

23 1 | The Quick Brown Fox

24 2 | jumps over the lazy dog

25 --- DOCUMENT END ---

26 ```

```
1  # Simple Line Editor (Minimal Edition) — Help
15 ## Commands
35 ### `INS <line#> <text>` — Insert a line
36 Inserts `text` at `line#`, shifting that line and everything below it
37 down by one. `line#` may be any value from `1` to `(current line count + 1)`
38 — using the highest value appends to the end.
39
40 ```
41 editor> INS 1 The Quick Brown Fox
42 Inserted line 1.
43 ```
44
45 ### `DEL <line#>` — Delete a line
46 Deletes the line at `line#`, shifting the lines below it up by one.
47
48 ```
49 editor> DEL 2
50 Deleted line 2.
51 ```
52
53 ### `EDIT <line#> <text>` — Replace a whole line
54 Overwrites the entire content of `line#` with new `text`. This is
55 different from `REPLACE`, which only swaps out a matching word or phrase
56 — `EDIT` replaces the whole line at once.
57
58 ```
59 editor> EDIT 1 A totally new first line
60 Updated line 1.
61 ```
62
63 ### `SAVE <filename>` — Save to file
64 Writes the document to a text file, one line per document line.
65
66 ```
67 editor> SAVE mydoc.txt
68 Saved to mydoc.txt.
69 ```
```

```
1  # Simple Line Editor (Minimal Edition) — Help
15 ## Commands

71 ### `LOAD <filename>` — Load from file
72 Reads a text file, **replacing** the current document with its contents.
73 If the file can't be opened, the current document is left unchanged.
74
75 ```
76 editor> LOAD mydoc.txt
77 Loaded 2 line(s).
78 ```
79
80 ### `FIND <phrase>` — Search
81 Searches every line for `phrase` (a word *or* multi-word phrase — anything
82 you type after `FIND` is treated as the whole search term) and reports
83 every matching line.
84
85 ```
86 editor> FIND Brown Fox
87 Line 1: The Quick Brown Fox
88 Found 1 match(es).
89 ```
90
91 If nothing matches, it still reports the count:
92
93 ```
94 editor> FIND zzz
95 Found 0 match(es).
96 ```
97
98 ### `REPLACE <target> <replacement>` — Find & replace
99 Replaces every occurrence of `target` with `replacement`, **across the
100 whole document** (this version does not support replacing on a single
101 line only). `target` must be a single word (no spaces); `replacement`
102 can contain spaces.
103
104 ```
105 editor> REPLACE the THE
106 Replaced 1 occurrence(s).
107 ```
```

1 # Simple Line Editor (Minimal Edition) – Help

You, 2 weeks ago • Add

15 ## Commands

139

140 ### `UPPER` / `LOWER` – Change case

141 Converts every line in the document to uppercase or lowercase.

142 Undoable with `UNDO`.

143

144 ```

145 editor> UPPER

146 Converted to UPPERCASE.

147 editor> LOWER

148 Converted to lowercase.

149 ```

150

151 ### `HELP` – Command list

152 Prints a one-line summary of all available commands.

153

154 ### `EXIT` – Quit

155 Exits the editor.

156

157 ---

158

159 ## Error Handling

160

161 The editor doesn't crash on bad input:

162

163 - Inserting/deleting/editing at a line number outside the valid range
164 prints an `Error/Usage:` message showing the valid range, and the
165 document is left unchanged.

166 - `INS` and `EDIT` require text to be given – an empty argument is
167 rejected with a usage message.

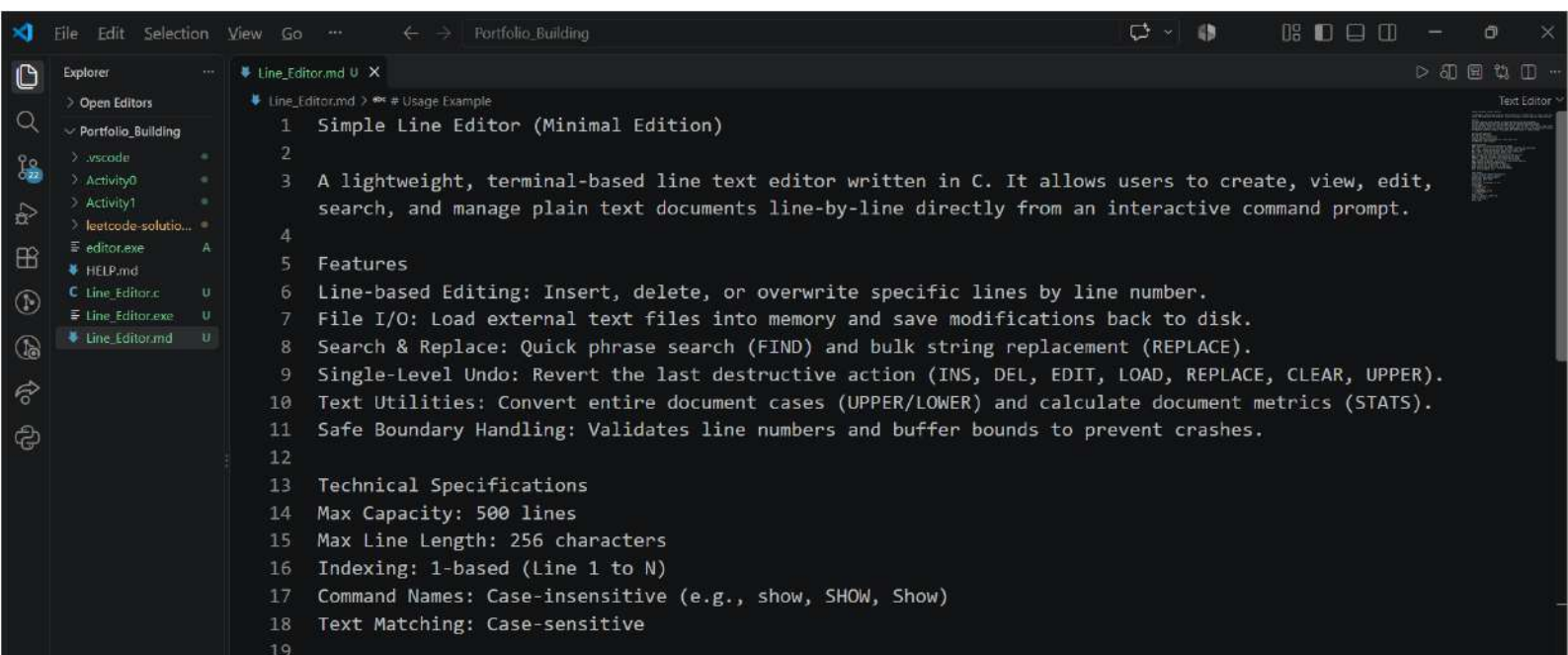
168 - `SAVE`/`LOAD` with a file that can't be opened prints an error instead
169 of crashing; a failed `LOAD` leaves the current document intact.

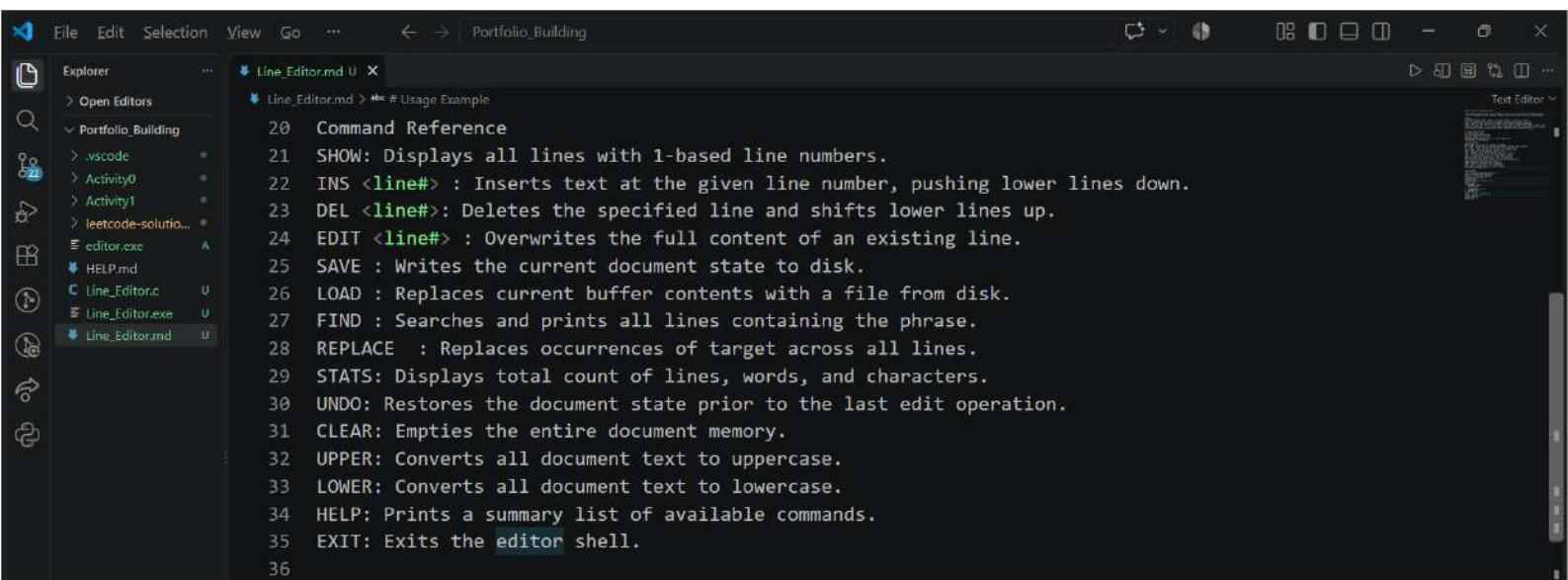
170 - `REPLACE` and `FIND` that match nothing still report a result
171 (`Replaced 0 occurrence(s).` / `Found 0 match(es).`) rather than
172 failing silently.

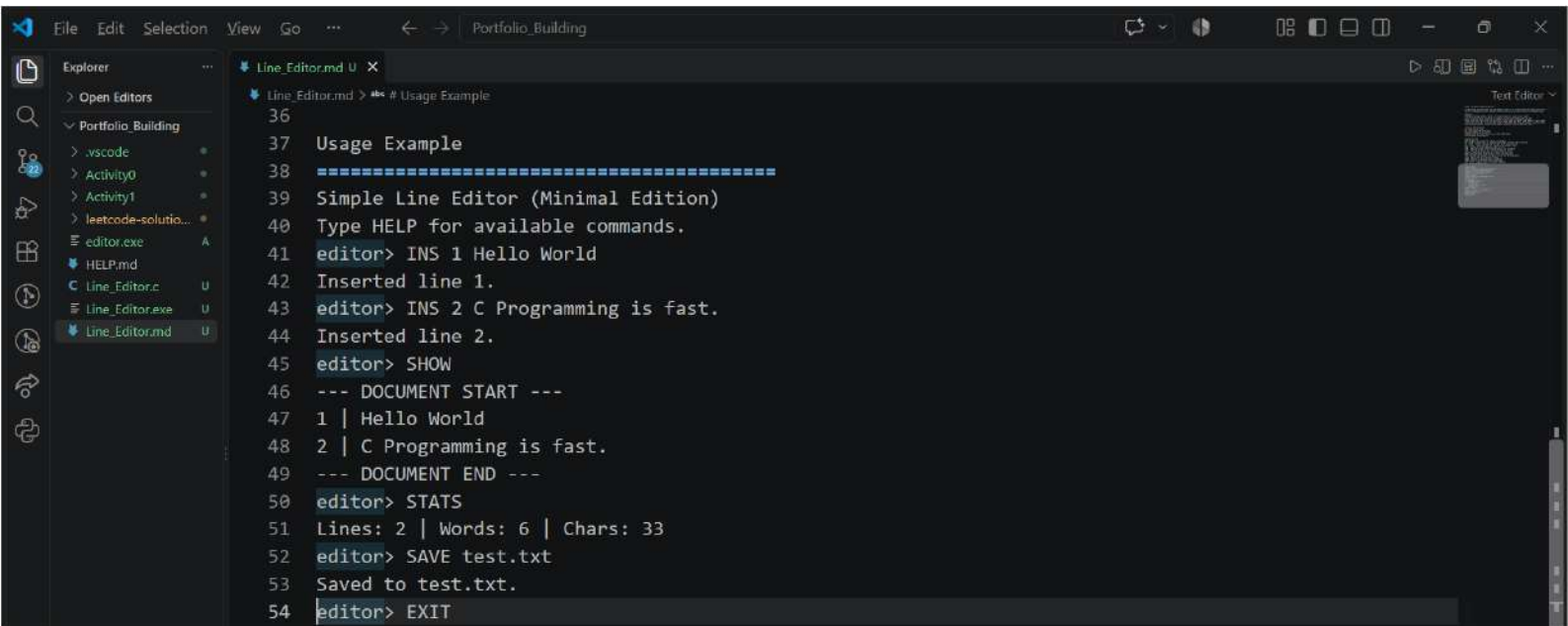
173 - `UNDO` with nothing to restore prints `Nothing to undo.` instead of
174 crashing.

175 - Unknown commands print `Unknown command. Type HELP for usage.`

176







```
File Edit Selection View Go ... Portfolio_Building
Line_Editor.md U X
Line_Editor.md > # Usage Example
36
37 Usage Example
38 =====
39 Simple Line Editor (Minimal Edition)
40 Type HELP for available commands.
41 editor> INS 1 Hello World
42 Inserted line 1.
43 editor> INS 2 C Programming is fast.
44 Inserted line 2.
45 editor> SHOW
46 --- DOCUMENT START ---
47 1 | Hello World
48 2 | C Programming is fast.
49 --- DOCUMENT END ---
50 editor> STATS
51 Lines: 2 | Words: 6 | Chars: 33
52 editor> SAVE test.txt
53 Saved to test.txt.
54 editor> EXIT
```